

USENIX

THE ADVANCED COMPUTING SYSTEMS ASSOCIATION

REAL: Emulating Control Plane at Simulator's Cost

Ze Xia, Hao Li, Jinyu Fu, Xin Wan, Yihan Dang, and Danfeng Shan,
Xi'an Jiaotong University; Li Chen, *harnets.ai;* Peng Zhang, *Xi'an Jiaotong University*

https://www.usenix.org/conference/nsdi26/presentation/xia

This paper is included in the Proceedings of the 23rd USENIX Symposium on Networked Systems Design and Implementation.

May 4–6, 2026 • Renton, WA, USA

ISBN 978-1-939133-54-0

Open access to the Proceedings of the 23rd USENIX Symposium on Networked Systems Design and Implementation is sponsored by



جامعة الملك عبدالله للعلوم والتقنية
King Abdullah University of Science and Technology



REAL: Emulating Control Plane at Simulator's Cost

Ze Xia[1], Hao Li[1], Jinyu Fu[1], Xin Wan[1], Yihan Dang[1], Danfeng Shan[1], Li Chen[2], Peng Zhang[1]

[1]*Xi'an Jiaotong University* [2]*harnets.ai*

Abstract

Validating control plane behavior and ensuring policy compliance in modern, large-scale networks is a critical challenge. Simulation-based approaches offer low computational and memory costs, but their level of abstraction fails to capture vendor-specific device behaviors, limiting their accuracy for real-world validation. In contrast, control plane emulation provides high fidelity by using unmodified router containers that preserve these vendor-specific details, but its excessive computational and memory requirements make it impractical for large networks. In this paper, we present REAL, a lightweight runtime that emulates control planes using unmodified router containers but at the cost of simulation-based approaches. REAL achieves this by simulating a lightweight data plane to accelerate boot-up, employing a two-phase scheduling policy to minimize cache inefficiencies during convergence, and enabling iterative convergence to reduce peak memory usage by partitioning the network. Our evaluation shows that REAL emulates a 1,000-node network 3× faster than state-of-the-art simulation while preserving vendor-specific behaviors, and can scale to 4,500 nodes on four commodity servers by shaving 8.3× memory.

1 Introduction

Today's networks span thousands of heterogeneous devices and must meet requirements such as routing waypoints and load balancing, while evolving rapidly due to failures and continual changes. To manage this complexity, operators rely on the control plane, *i.e.*, routing protocols (BGP, OSPF, *etc.*) and routing policies, yet faults there can cause severe data plane outages. In practice, operators feed data plane configurations to verification tools to verify compliance with network intent [47, 51]. Therefore, before deploying or modifying the network, operators must answer a key question: given specific control plane configurations, what will the data plane, *e.g.*, FIBs and ACLs, look like?

A common approach is to *simulate* control plane behavior [20, 25, 40, 50], which parses topology and configuration files, then runs routing protocol simulations to generate the FIB and ACLs. However, whenever vendor implementations change or protocols evolve, the simulator has to be updated accordingly with considerable human effort, including a deep understanding of the device manuals or protocol semantics and thousands of lines of code to parse configurations and simulate routing logic. What is worse, this process is error-prone, because the manual can contain ambiguity, and the device may perform vendor-specific behaviors (VSB), undocumented behaviors (UDB), or even faulty behaviors due to software bugs [27, 33, 48]. As a result, control plane simulation is subject to high fidelity risk.

Alternatively, one can *emulate* the control plane behaviors by running and connecting real router images with containers and/or virtual machines (VMs) [2, 3, 8, 11, 16, 23, 27, 31, 33, 36]. As these images are provided by vendors, operators do not need to re-implement routing protocols, all VSBs, UDBs and even software bugs are thus faithfully preserved. While ideal for fidelity, the practical applicability of control-plane emulation is limited by its high computational and memory cost. For example, CrystalNet takes 30 minutes to emulate a 4K-node network, even with 500 virtual machines, which is equivalent to 2K cores and 4TB of memory [33]. We note that nowadays vendors tend to provide their router images as containers instead of VMs [6, 7, 14], but still do not provide sufficient performance as the simulator does. As a reference, Batfish can simulate the above 4K-node network within 5 minutes on a 10-core laptop [20].

At first glance, the high cost of emulation seems to stem from heavy images and inefficient protocol implementations. Our profiling points to a different root cause: The operating system's *default runtime* is ill-suited for control plane emulation. First, data-plane virtualization is heavyweight; our results show that container boot-up and network creation dominate the emulation time. Second, the fairness-oriented scheduler frequently preempts routing message processing, causing high cache misses and increasing convergence time. Finally, the default runtime keeps all routers resident in memory even when they are temporarily idle, inflating peak memory usage.

Detailed evidence is presented in §2.3.

These inefficiencies reflect a mismatch of goals. The default container runtime optimizes for the data plane, *e.g.*, OS-level virtualization for full-stack fidelity, fairness-scheduling for low per-packet latency, and all-container residency for fast response. In contrast, emulation is driven by control plane semantics: the content of routing messages and the eventual convergence state. For example, BGP is an application layer protocol whose correctness depends on reliable delivery of UPDATE/KEEPALIVE, not on TCP/IP retransmission mechanism; likewise, convergence correctness is about the final stable routes, not the exact order of intermediate updates. Such a mismatch makes much of the runtime work unnecessary or suboptimal for emulation.

In this paper, we replace the default runtime with a *RuntimE for control plAne emuLation (REAL)*, which runs unmodified router containers, while optimizing their execution guided by control-plane semantics. The key insight is that REAL *emulates the control plane but simulates the runtime*. The rest of the paper mainly focuses on BGP networks, but our methods can be generalized to other protocols (see Appendix E).

The concrete contributions are listed below.

Simulated data plane to accelerate boot-up (§4.1). REAL simulates a lightweight data plane tailored for control-plane emulation. It replaces expensive `veth-pair`s with Unix domain sockets (UDS), transforming TCP/IP connections into inter-process communication. REAL preserves upper-layer semantics under this replacement, allowing router containers to run unmodified.

Scheduled execution to minimize cache misses (§4.2). REAL employs a two-phase scheduler to optimize the execution order of router processes. During peer connection establishment, REAL ensures that all connections are successfully set up before any route messages are sent. For route message exchange, REAL adopts a run-to-`Idle` scheme, where each router is executed until it processes all received messages and sends all pending updates.

Iterative convergence to reduce peak memory (§4.3). REAL iteratively activates only a subset of routers, while snapshotting the rest inactive ones. To maintain convergence consistency, REAL carefully partitions the network and ensures that the routing messages propagate correctly across the boundary. Instead of relying on slow, disk-based snapshot mechanisms, REAL reconstructs the router state by restarting inactive routers and re-injecting the history messages, allowing them to quickly re-converge using in-memory state.

We evaluate REAL with large-scale network topologies and real router containers (§6). The results show that REAL achieves comparable or even better performance. Specifically, REAL can emulate a 1,000-node network in *3× less time* than Batfish and scale to emulate a 4,500-node network on four commodity servers (each with 64 cores and 256 GB memory) reducing memory usage by *8.3×*.

2 Motivation, Observations and Potentials

In this section, we discuss why control plane emulation matters to network management (§2.1) and why workarounds to mitigate the cost of emulation fail (§2.2). Next, we set up a testbed to profile the performance of the emulation approach, which pinpoints the causes behind its high costs and reveals the potential to address these issues (§2.3).

2.1 Why Emulation Matters?

To reliably manage large and complex networks, operators ultimately need to understand the data plane, *e.g.*, which prefixes will be installed in FIBs, how ACLs will be enforced and whether loops or black holes will appear. For this reason, practical systems first compute the data plane from the control plane [20, 25, 27, 33] and then verify the forwarding state [47, 50, 51].

The key challenge is fidelity, especially in the presence of VSBs. Simulation-based CPVs re-implement routing protocols, but the models are inevitably incomplete. Different vendors, and even different firmware versions, often interpret the same command differently. A well-known example is remove-private-AS: one vendor removes all private AS numbers, while another stops at the first public AS [48, 49]. Such differences have caused real outages in production networks [27, 33]. Simulation tools cannot anticipate all these VSBs, nor can they capture UDBs or latent software bugs, and therefore lead to lower fidelity. By running real router images, control plane emulation naturally preserves these details and has been shown to uncover both VSBs and software flaws in vendor implementations [27, 33]. This fidelity makes emulation uniquely reliable for deriving data plane configurations.

2.2 Workarounds to Costly Emulation

While prior works admit the high cost of control plane emulation, some might argue that there is no need for further optimization, or that an easy solution already exists. Specifically, we answer the following questions to show the necessity of further boosting the emulation's performance.

(1) Can we emulate only a small portion of the network? Prior works attempt to reduce the scale of emulation to lower resource costs. CrystalNet finds a safe static boundary for emulation, so it only needs to emulate 10% of the network [33]. Crescent emulates all the inter-DC (core) devices and only samples a small portion of intra-DC (non-core) ones, since the latter barely causes network incidents. This further reduces the scale of the network by 98% [27]. However, the safe boundary may not exist in the inter-DC network, which means that all core devices must be included for emulation; and each sampled non-core device would expand into tens of routers to be connected to the core network [27]. In sum, thousands of routers still remain to be emulated [27, 33].

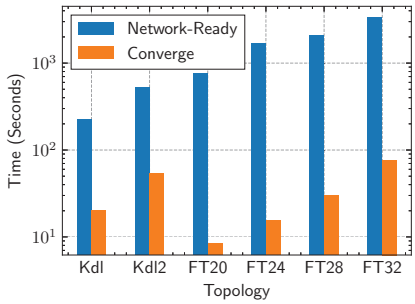


Figure 1: Time breakdown for emulation with full-fledged data plane.

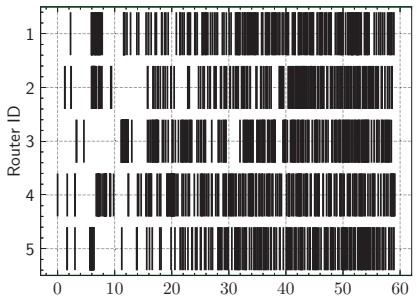


Figure 2: Event chart for sampled routers with fair scheduling.

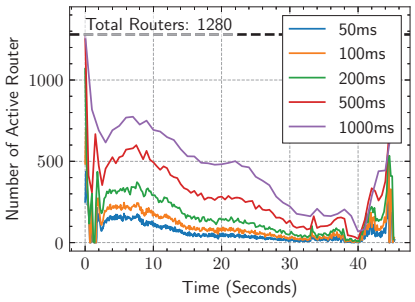


Figure 3: The number of active routers grouped by different time windows.

(2) Can we scale out control plane emulation to distributed servers? Previous works show that a cluster would largely boost the emulation speed [27, 33], but still lacks scalability. First, due to the runtime inefficiencies in the single server, scaling out results in a huge number of cores and memory, *e.g.*, Crescent uses a 4-server cluster with 512 cores and 2TB memory to emulate only the core devices [27]. Second, scaling-out demands extra costs for inter-server communication. For example, Crescent finds that the creation of cross-server bridges costs $O(n^2)$ time, where n is the total number of bridges. One possible mitigation is to split the topology into more servers, to reduce the number of links between servers [33, 38]. However, this wastes much more memory for each host OS, *e.g.*, more than 50GB for Fattree $k = 32$ divided into 4 hosts. In sum, scaling-out is promising for large-scale emulation but can only be effective if both single- and cross-host cost are significantly reduced.

(3) Can we have an always-online emulator to avoid frequent full emulations? Control plane emulation inherently supports incremental modeling: when a link fails or a configuration file changes, only a small number of routing updates need to be recomputed. This leads to the idea of an always-online emulator, where the costly full emulation occurs only once during the cold start, and subsequent events are handled incrementally. However, an always-online emulator continuously occupies cores and memory, and can only serve one emulation task at a time. If operators want to test a new topology or protocol, they must maintain multiple emulator instances. For example, Crescent runs several instances of its 4-server cluster to handle concurrent emulation requests [27].

2.3 Observations and Opportunities

We profile the control plane emulation with a testbed (64 cores and 256GB memory), running real router images (containerized FRRouting connected with `veth-pair`). We then present the high cost of emulation, and reveal opportunities for designing a new runtime to overcome such cost.

Data-plane setup dominates the emulation time; control-plane semantics allow a lightweight replacement.

Observation: Container boot-ups contribute the most to the emulation time. We break down the emulation time into two phases: Network-Ready, where containers are loaded into memory and the network (*e.g.*, links, bridges, IPs) is initialized; and Converge, where routers establish BGP sessions and reach convergence. Figure 1 shows that Network-Ready dominates the time usage, *i.e.*, at least one order of magnitude longer than the Converge phase. This overhead stems from serialized creation of `veth` interfaces and bridges under global kernel locks (*e.g.*, RTNL [44]), which scales with link count. Similar overhead has been observed in production environments at ByteDance [27] and Alibaba Cloud [35].

Opportunity: Data plane fidelity is unnecessary to control plane emulation. Routing protocols work at the application layer, which means the L2–L4 stack is irrelevant to the routing semantics. Some may argue that the fully functional data plane enables debugging tools like `ping` and `traceroute`. However, these tools are unnecessary in control plane emulation, as existing data plane verification tools [47, 51] can validate route tables directly without requiring actual packet forwarding. Another concern is that an insufficiently faithful data plane may alter control-plane behavior. For example, routing protocols may react differently to packet drops, congestion, or forwarding failures that depend on data-plane execution. However, even with a fully functional emulated data plane, its software nature still prevents it from faithfully reproducing the behavior of physical routers or capturing hardware-related effects. Thus, the perceived "authenticity" of the emulated data plane is limited and does not justify the additional complexity for emulation. In short, we argue that the control plane operates unmodified as long as it can receive the same route messages. This allows us to realize a lightweight channel for the communication instead of a full virtualized data plane, eliminating the heavy boot-up overhead.

Fair scheduling hurts cache locality; order-flexibility enables coalesced execution.

Observation: Fair scheduling causes high cache misses. To measure the context switch, we draw an event chart for router processes, where each vertical line means the CPU is serving the router. Figure 2 shows the event chart for 5 sampled routers in Fattree $k = 32$, where the default runtime frequently switches CPU context between routers. We also measure the cache misses of emulation and Batfish respectively. We report that the last-level cache (LLC) miss rate of

emulation approach goes beyond 70% for Fattree $k = 32$.

Opportunity: Routing protocols permit flexible message scheduling. Routing protocols are designed to tolerate arbitrary message arrival orders and batching. For a large class of operator configurations that enforce deterministic tie-breaks and policy outcomes, different fair schedules converge to the same stable routing solution (*i.e.*, identical best routes/-FIB entries), even if intermediate message interleavings differ [22, 39]. When the protocol and configuration intentionally admit multiple semantically equivalent outcomes (*e.g.*, ECMP), different schedules may select different yet still valid next-hop sets, consistent with the protocol specification and operator constraints [34, 46]. Therefore, it is safe for REAL to control message delivery schedules to reduce context switches and cache misses while yielding a correct routing solution.

Most routers are temporarily idle; shaped scheduling can reduce peak residency.

Observation: Many routers are temporarily idle through convergence. We count the active routers, *i.e.*, routers that receive or send messages, within different time windows for Fattree $k = 32$. The measurement starts when all containers are network-ready. As shown in Figure 3, except for a short peak, there are only half (or fewer) routers alive in most cases, while the default runtime keeps all 1280 routers in memory throughout the convergence process.

Opportunity: Routers can be deliberately scheduled to reduce the peak. Control-plane semantics allow us to deliberately create and prolong idleness: since convergence is order-flexible, we can schedule updates in a way that minimizes the peak number of active routers, while inactive routers can be snapshotted to reduce memory usage. For example, in a Fattree topology, if we let the core nodes communicate with only one pod at a time, all other pods naturally remain idle because they cannot receive routing information through the core. This scheduling policy makes it possible to extend each pod's idle time to roughly $(N-1)$ times the convergence time of a single pod, where N is the number of pods. This demonstrates that idleness is not just an observation but a controllable resource: by shaping message flow, we can engineer longer and fewer active periods for routers, thereby lowering residency and shaving the peak.

3 REAL Overview

In this section, we provide an overview of the workflow of REAL (§3.1), and then pose the ongoing challenges that obstruct its practicality (§3.2).

3.1 REAL Workflow

REAL emulates the network with containerized router images. To replace the default OS runtime without modifying images, REAL employs a *shim* layer that intercepts the communication between containers and the OS, such as I/O and route

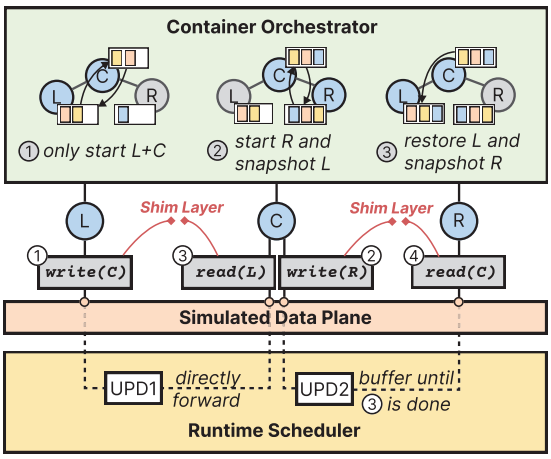


Figure 4: Workflow of REAL. L, C and R are unmodified containerized router images. REAL connects the routers with a *Simulated data plane (SimDP)*, schedules them with a *Runtime Scheduler*, and controls their convergence process with the *Container Orchestrator*.

installation. This is done by loading a custom library into each router process via `LD_PRELOAD` [26], which overrides default system calls without modifying router images. Based on the shim layer, REAL implements three key mechanisms: (1) a simulated data plane (SimDP) to connect the containers, (2) a runtime scheduler to control the execution order of the containers, and (3) a container orchestrator to snapshot and replicate containers during convergence.

In the following, we use a simple linear topology $L-C-R$ to walk through the workflow of REAL, as shown in Figure 4.

Communicating with simulated data plane. The shim layer redirects socket-related syscalls like `connect()`, `read()` and `write()` to SimDP. The redirected system calls then communicate through SimDP using inter-process communication (IPC). Specifically, when L calls `connect()` to C, its redirected version will create an IPC connection to SimDP. Then, SimDP will attempt to connect to C and will complete the pairing when C calls `accept()`. Finally, SimDP bridges these two connections to relay the communication between L and C. Similarly, C and R would also build two IPC connections with SimDP for message exchanging. In this way, the kernel network stack and virtual link setup (`veth-pairs` and bridges) are bypassed, so the boot-time overhead from creating these data-plane components is avoided.

Controlling the execution with runtime scheduler. REAL schedules the execution of routers by controlling the order and timing of delivering routing messages. This is possible because the main processing logic of a router is driven by incoming routing messages; without new messages, a router soon becomes idle and releases the CPU. For example, in Figure 4, L sends an UPDATE message (UPD1) to C (①), while C sends an UPDATE message (UPD2) to R with its own routes (②). REAL buffers UPD2 until C processed UPD1

(③), then delivers UPD2 to R (④). Thus R does not contend for the CPU while C is busy processing UPD1, which reduces context switches and cache misses.

Converging iteratively with container orchestrator. Instead of keeping all routers resident in memory, REAL snapshots idle containers and restores them only when they have work to do. For the simple linear topology, REAL starts L and C only (①). And after these two routers exchange their routes, REAL snapshots L and starts R. Since C has all routes from L, it can propagate all routes to R (②). Finally, REAL snapshots R and restores L to propagate R's route back to L (③). In this way, the network iteratively converges, *i.e.*, all routers have all routes, while the peak memory is reduced from $(L+C+R)$ to $\max(L+C, R+C)$.

3.2 Ongoing Challenges

While REAL makes an efficient runtime possible, several practical challenges remain.

Consistent semantics and efficient scaling-out. Beyond socket send/receive, routers rely on OS for setting up interface and link status, installing routes to the kernel RIB/FIB, *etc*. Therefore, SimDP must preserve the observable behavior of a wide range of system calls and argument combinations, *e.g.*, `socket`, `bind`, `getsockopt`, `accept`, `ioctl`, `ppoll`, and related helpers. Missing a call path or altering return values can cause inconsistent behavior. Besides, the super-large-scale emulation may require multiple hosts to coordinate, which means REAL also has to simulate the communication between cross-host containers without compromising its scalability.

Comprehensive scheduling. Figure 4 shows that limiting concurrency can reduce context switches during route processing. However, during large-scale BGP initialization, overly constrained concurrency can slow down session establishment (BGP OPEN/KEEPALIVE). Note that these two phases are naturally mixed up, *i.e.*, when some routers are establishing connections, others may exchange route messages. As a result, we need a comprehensive scheduling policy that adapts across two phases: (1) allow higher parallelism for shorter connection establishment; and (2) throttle delivery during route exchange to reduce preemption.

Efficient snapshot and effective partition. Iterative convergence reduces peak memory, but it introduces extra snapshot and restore overhead. When REAL tries to reduce the peak memory in a large topology, thousands of routers would be snapshotted and restored in a short period, each occupies tens of MB of memory, along with per-container metadata (*e.g.*, file descriptors and socket state). That hundreds of GB of state and per-container metadata handling overhead make the memory-to-disk snapshots infeasible in practice. We quantify and analyze this cost in §4.3. Moreover, arbitrary partitioning may result in a large number of iterations, especially when the route information traverses multiple partitions. This leads to two challenges: (1) an efficient snapshot mechanism that avoids slow memory-to-disk operations; and (2) an effective partition strategy that minimizes the number of iterations.

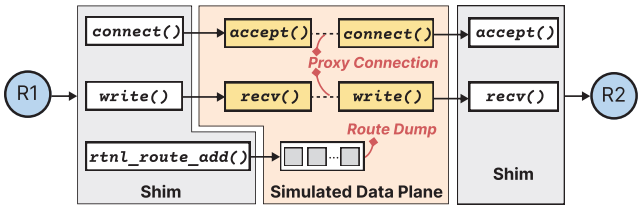


Figure 5: Simulated data plane in REAL for creating TCP connection, sending route messages and installing route table.

4 Detailed Design of REAL

In this section, we detail the design of REAL to address the ongoing challenges posed in §3.2.

4.1 Simulating Data Plane Behaviors

We describe how REAL simulates data plane behaviors to support realistic network communication in the control plane including (1) simulating TCP and NETLINK sockets in a single machine, (2) simulating timing behaviors, and (3) coordinating multiple hosts for large-scale emulation.

Simulating TCP and NETLINK sockets. REAL chooses a classical IPC technique, Unix domain sockets (UDS), to simulate the peer connections, which provides the same reliable, ordered stream semantics as TCP, making it sufficient for exchanging routing messages in the control plane. Additionally, UDS can be easily integrated with Linux asynchronous event frameworks like `ppoll()` and `epoll()`, which could be tedious to implement if a shared-memory approach is adopted. Note that UDS is an implementation choice made for simplicity and practicality, not a fundamental requirement. Alternatives (*e.g.*, an eBPF-based IPC [13] or a user-space TCP stack [9, 29, 52]) could also work, as long as they preserve TCP's reliable, in-order stream semantics and integrate cleanly with the event-driven execution model.

To detect network communication attempts, we redirect the `socket()` libc call that tries to create a TCP connection to a corresponding `socket()` call into `libc` that creates an UDS. Other `socket()` calls are simply passed to the original `libc` implementation. Later calls on those TCP sockets are either redirected to corresponding UDS calls, *e.g.*, we translate TCP addresses in `bind()` calls into UDS addresses, or directly handled in the shim, *e.g.*, the shim directly synthesizes the return value of `getsockopt()` without invoking the kernel. Subsequent `read()`/`write()` calls on the sockets are directly passed to the kernel. Table 2 in Appendix B shows intercepted syscalls to simulate TCP sockets.

NICs announce ownership of IP addresses and prefixes using NETLINK sockets. The shim redirects these NETLINK

socket calls to SimDP to simulate their behaviors. NETLINK sockets are also used to install routes into the kernel. Since control plane emulation does not require actual packet forwarding, the shim simply dumps the routes to SimDP for later exporting.

We use Figure 5 to show how REAL creates TCP connection, relays route messages and installs the selected routes. When $R1$ tries to `connect` $R2$, the shim would redirect the function to SimDP, which `accepts` the connection and `connects` to $R2$. A proxy channel is built when $R2$ eventually `accepts` the connection, which relays later messages through the intercepted `write` and `recv`. When $R1$ installs a route by `rtnl_route_add`, SimDP would record the route without passing it to kernel. In this way, all communication happens entirely in user space, bypassing the heavy-weight kernel.

Simulating time behaviors. In emulation, the timers of BGP KEEPALIVE face a practical challenge: multiple router containers compete for limited CPU resources while real time advances continuously. A router might miss a KEEPALIVE deadline not because of network issues, but because it was starved of CPU cycles.

REAL addresses this through per-container simulated clocks that decouple simulated time from wall time. Routing protocols observe time in two distinct ways: incidental observations, *e.g.*, timestamps for logging, which typically use `gettimeofday` or `clock_gettime` for timestamping; and protocol-critical observations, *e.g.*, KEEPALIVE timeouts, which are usually implemented as blocking waits via syscalls such as `select`, `poll`, or `epoll_wait`. For the first, we advance the simulated clock with a very short time slot, say 1μs, to prevent spurious timeouts regardless of repeated time observations. For the second, we advance its simulated clock directly to the next meaningful event, *e.g.*, a timer expiration or message arrival. We discuss the correctness of this design in Appendix F.

Coordinating multiple hosts for emulation. We extend the single-host abstraction to multiple hosts. Each host runs a SimDP that mediates all BGP I/O for local containers. Routers talk to the local SimDP over UDS, exactly as on a single host. Hosts connect their SimDPs using a regular TCP link.

For cross-host peers, the end-to-end path is router→local UDS→SimDP→TCP→SimDP→local UDS→router. When a router on host A sends a message to a peer on host B, SimDP on A encapsulates the raw BGP message together with a small shim header that carries the source and destination router IDs. It sends this blob over the SimDP-to-SimDP TCP connection. On receipt, SimDP on B reads the router IDs, selects the correct local UDS, strips the encapsulation, and delivers the original BGP message to the target container. The router-facing interface remains unchanged.

The single TCP connection design avoids creating per-neighbor OVS links across hosts, which has been proven quite costly [27]. Moreover, multiplexing over a single TCP connection is feasible because emulation is I/O-light: our evaluation shows that even with 40,500 concurrent BGP sessions, the shared TCP connection uses only 100 Mbps of bandwidth (see §6.3 for details).

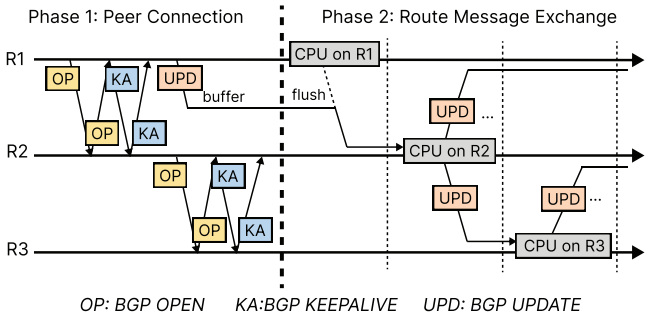


Figure 6: Two-phase scheduling: (1) peer connection and (2) message exchange. BGP UPDATEs in the first phase are delayed until the second phase; then a router keeps running on the CPU until it does not have any message to process.

4.2 Two-Phase Scheduling

The runtime scheduler can control the concurrency of execution by buffering and flushing the incoming packets in SimDP. However, the desired concurrency varies across the different convergence phases, as detailed below.

(1) Peer connection establishment. During this phase, routers establish connections with their neighbors. In BGP, every neighboring establishment only takes two OPEN messages and two KEEPALIVE messages, and the processing on each message is quite lightweight. Consequently, we desire high concurrency in this phase, such that the whole network can be built up as soon as possible.

(2) Route message exchange. During this phase, routers frequently send and receive routing messages. As presented in §2.3, frequent scheduling across different routers disrupts memory locality, leading to unnecessary cache misses. As a result, we should limit concurrency during this phase to maintain high cache efficiency.

To this end, REAL employs a two-phase scheduling policy that separates the connection establishment and route exchange, respectively addressing their concurrency needs.

Peer connection until `Ready`. During the peer connection establishment phase, REAL allows nodes to send only connection-setup messages while buffering all messages that could update routing tables. This continues until all routers establish connections with their neighbors, *i.e.*, `Ready` to exchange route messages. Since no router can receive any route updates, the overall workload is low, which allows CPUs to serve other connection requests. As shown in the left part of Figure 6, $R1$ and $R2$ successfully establish a BGP connection, but their route updates are buffered. So the connection between $R2$ and $R3$ can be established without much delay. This design enables each router to process a large batch of route

updates during later convergence.

To achieve the above process, the runtime scheduler has to (1) correctly forward/buffer the OPEN/UPDATE messages; and (2) deterministically detect `Ready` state for each router.

Specifically, the runtime scheduler leverages the intercepted `write()` system calls to inspect outgoing packets, and maintains a simplified BGP state machine for each connection. The state machine inspects OPEN, KEEPALIVE and UPDATE messages, and takes care of the exceptions like simultaneous connection attempts, timeouts, identifier conflicts, *etc*. Before all the BGP connections in the network are established, for all messages prior to route updates (OPEN, KEEPALIVE messages), `write()` will directly send them through UDS; while for all UPDATE messages, `write()` will immediately return, and buffer their content in the scheduler. Once all BGP sessions for a router are established, the router is deemed as `Ready`. When all routers are `Ready`, the scheduler will flush the previously buffered UPDATE messages, and the route exchange phase begins.

Run-to-`Idle` during route exchange. During the route exchange phase, REAL monitors each router's state, and keeps it running as long as it has messages to process. When it becomes `Idle`, *i.e.*, handled all received messages, the entire container of the router is scheduled off the CPU. This run-to-`Idle` approach ensures that each container processes all pending messages before being descheduled, minimizing unnecessary context switches. As shown in the right part of Figure 6, when CPU is dedicated to $R1$, $R2$ cannot respond to any of the updates. After $R1$ has processed all route updates, CPU switches to $R2$, and now it can process the previously incoming messages and send its updates to neighbors.

The key challenge is detecting `Idle` without understanding the router code. We advocate that we don't need to understand exact application-level semantic `Idle`-ness, instead we can detect CPU usage `Idle`-ness from the runtime side. We observe that most router implementations share a common architectural pattern: they use event-driven loops that ultimately wait for external events through system calls like `select()`, `poll()` or `epoll_wait()`. This creates a natural signal of `Idle`-ness: when there are no pending events, these function calls block, indicating that the calling thread is `Idle`. When these event-checking functions are invoked, REAL performs the same event checks with a zero timeout (*i.e.*, without blocking). If no events are returned, the original call would have blocked, and REAL therefore treats the calling thread as `Idle`.

However, router containers run multiple processes and threads, *e.g.*, FRRouting runs bgpd and zebra, each with multiple threads. Thus, detecting a single idle thread is insufficient. To coordinate across processes, REAL maintains a shared atomic counter per container: each thread increments it at startup and decrements it when entering `Idle`. When the counter reaches zero, indicating the container-wide `Idle` state, the shim notifies the scheduler to deschedule this container and activate another.

The `Idle`-ness detected by our approach may not fully align with application-level semantic idleness. However, REAL only needs coarse-grained idle identification to reduce unnecessary preemptions and improve cache locality, rather than precise control-plane semantic quiescence detection. We claim that our design is sufficient to achieve this goal, as shown in our evaluation (§6.3).

4.3 In-Memory Re-converging over Partitions

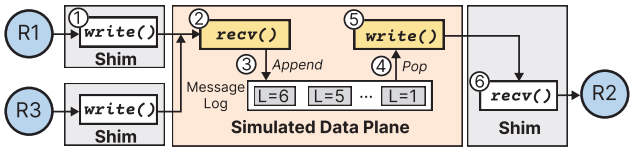
As discussed in §3.2, iterative convergence requires lots of snapshots and restores, which makes the traditional memory-to- disk mechanism impractical. Besides, different partition strategies would affect the number of iterations. We describe how REAL addresses these two challenges in the following.

Snapshot and replication with in-memory reconvergence. Traditionally, snapshotting containers involves saving their memory state to disk and restoring it back to memory during recovery, *e.g.*, CRIU [5]. Even considering a fast SSD with several GB/s read/write speed, each iteration would cost several hundreds of seconds. Worse, emulation deals not with a single large container but with 1K small containers. The metadata of each container (*e.g.*, file descriptor, socket state) must be processed individually, which costs hundreds of milliseconds for each operation [42]. The overall overhead of each iteration therefore becomes prohibitively high.

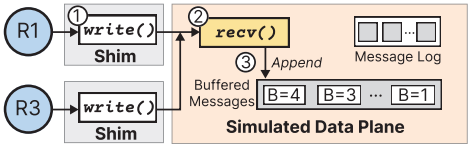
In contrast, we advocate that a byte-to-byte snapshot, *e.g.*, identical memory layout and kernel status, is not necessary in our scenario. Rather, we only need to ensure the *router status* such as route table and route attributes remains the same after replication. This enables a simple snapshot solution: restarting containers and allowing them to re-converge, which could potentially outperform disk-based snapshots, because stopping containers incurs negligible overhead and reconvergence occurs entirely in memory.

While possible, simply restarting a router would force its neighbors to drop the related routes, harming the correctness. Recall the example where L is shut down in the second step of Figure 4. C would remove all routes learned from L after detecting that L is offline, so the propagation to R would fail.

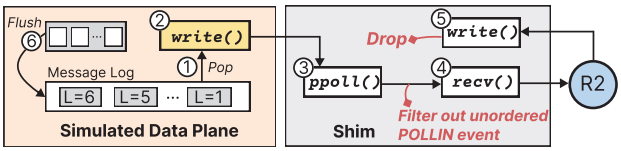
To avoid unintentional withdrawals, REAL exploits SimDP to record and replay the route messages, so as to restore the router status. As shown in Figure 7a, SimDP records every route message sent to each router, and keeps them in order. When the router $R2$ is offline, SimDP keeps the connections to all neighbors of $R2$, echoing the KEEPALIVE messages, and buffers the messages sent to $R2$, so that neighbors do not know that $R2$ is offline, as shown in Figure 7b. When restoring router $R2$, SimDP will replay the recorded messages to $R2$, as shown in Figure 7c. To ensure that internal events of $R2$ happen in exact order with the original trace, intercepted `ppoll()` would filter out all unordered `POLLIN` events. All messages sent by $R2$ during the replay phase are first validated against those observed in the original execution, and



(a) Snapshot $R2$ by recording its received messages.



(b) Keeping connection and buffering messages while $R2$ is offline.



(c) Restoring $R2$ by replaying the recorded messages.

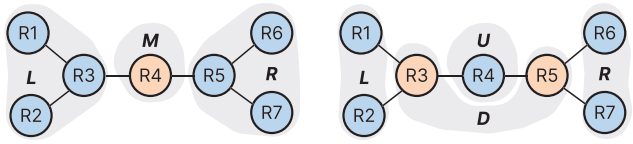
Figure 7: Snapshot and restore router status with message replay. (a) When $R2$ is online, simulated data plane records all route messages sent to $R2$ before relaying them. (b) When $R2$ is offline, simulated data plane buffers all route messages sent to $R2$ without touching the message logs. (c) When restoring $R2$, SimDP replays the recorded messages to $R2$, while filtering out unordered `POLLIN` events in `ppoll`.

are then suppressed to avoid perturbing neighbors. Since $R2$'s route table and attributes come solely from the routing messages, the replay should eventually restore its status to the moment it gets offline. This is validated in §5.1. Now SimDP would record the message logs as in Figure 7a, and flush the messages buffered while $R2$ is offline.

Network partition with bridge nodes. The ideal partition should achieve two objectives: (1) minimize peak memory usage and (2) minimize the number of iterations.

Given the ability of the message buffer and replay, REAL can arbitrarily snapshot any router without breaking correctness. However, minimizing peak memory alone does not yield a feasible plan. As an extreme, one could activate a single router at a time, snapshot it, then proceed to the next; this drives peak memory down to that of one router, but the number of iterations explodes because routes must traverse multiple components repeatedly. Hence we first design to minimize iterations, and then balance peak memory under that constraint.

Reconsider the example in Figure 4, which only requires two iterations for full convergence. This indicates that a small set of always-on nodes (C in Figure 4) can relay and compose routing information across different parts of the network, so that other routers (L and R) need not be online simultaneously. Intuitively, if every end-to-end path that crosses different regions can be funneled through such a set, each region can upload its routes to this set in one iteration and download



(a) (b)

Figure 8: Using vertex cut to partition topology. (a) A good cut that separates the optimal path into two segments, ensuring the convergence in two iterations. (b) A bad cut that produces three segments, risking a third iteration for full convergence.

missing segments in the next. Graph-theoretically, this intuition corresponds to choosing a set of *bridge nodes* that forms a vertex cut C for topology G: removing C disconnects G into components $S_1, \ldots, S_N$, and any path that spans multiple components must traverse C.

Optimality and limitation. REAL keeps C always online, while iteratively activating $S_1 \ldots S_N$. In the first iteration, each component exports its routes to C; in the second iteration, components combine what they learn from C and push back the completed results. For example in Figure 8a, the cut M would propagate all information from the outside world (*i.e.*, routes in R) back to L in the second iteration. In this way, two iterations make the whole network converge.

However, two iterations are not guaranteed in all cases. When a pair's (policy-)shortest path is segmented by C more than once, composition can miss in the second iteration simply due to timing: a component that should supply the final segment may come online before C has learned the upstream segment from the opposite side. Figure 8b illustrates this adverse pattern: the end-to-end path crosses C twice, and with an unfavorable order in the second iteration (*e.g.*, L before U), C has not yet accumulated the needed segment when L is active. As a result, L cannot complete composition in that visit, and a third round with L online becomes necessary.

To conclude, using a vertex cut as the bridge would largely reduce the number of iterations, but the upper bound depends on how many times shortest paths are segmented by C.

Problem statement and proposed heuristic. Based on the above analysis, our goal becomes finding a vertex cut as the bridge which (1) minimizes the peak memory, and (2) minimizes the number of segments for any optimal path.

For the first goal, assume each router occupies similar memory, the peak memory is $M(C) = |C| + \max_i\{|S_i|\}$. For the second goal, as we cannot obtain the "true" optimal path beforehand, we assume the shortest path as a surrogate, and define a segmentation score $\mathrm{Seg}(C)$, which is the number of segments C produces for all shortest paths. Our goal then becomes finding C with minimum $M(C)$ and minimum $\mathrm{Seg}(C)$.

Solely considering $M(C)$ is actually an extension to the well-known Minimum Balanced Cut Problem, which has been proved to be NP-Hard [21]. REAL exploits a heuristic algorithm to find a near-optimal solution. The intuition is to find

the "narrow waist" of the network, removing which would effectively create separate components. To locate such nodes, we measure each node with *betweenness centrality* [45], *i.e.*, how many pairwise shortest paths traverse the node. Nodes with high betweenness centrality tend to lie on many shortest paths and thus are natural bridge candidates. We iteratively build C by adding high-betweenness nodes, then evaluate the partition $G \setminus C$ via a combined score from $|C|$, $\max_i |S_i|$ and $\mathrm{Seg}(C)$, and keep the best under a budget.

On structured topologies (*e.g.*, Fattree), the heuristic selects core-layer nodes and recovers the optimal bridge (see Appendix C); on WANs, it gravitates to backbone hubs and yields compact cuts with low segmentation in practice.

5 REAL in Action

We prototype REAL with ∼10k C++ LOC for the shim layer and SimDP, handling 38 `libc` functions. REAL currently supports running router containers from three vendors: FRRouting [10], BIRD [15] and cRPD from Juniper [7]. The implementation of REAL is open source [17].

In this section, we validate the correctness of REAL (§5.1), and share one of our experiences of using REAL (§5.2).

5.1 Correctness

To validate that REAL preserves BGP correctness despite its aggressive optimizations, we conduct extensive experiments with REAL and compare the outcome with the default Linux runtime. For topologies, we mainly involve Fattree with different k and WAN topologies from Topology Zoo [1]. For configurations, beyond basic reachability, we evaluate complex policies including route reflection, ECMP load balancing, community-based filtering, AS-path prepending, and MED attributes that create intricate routing dependencies.

REAL produces identical (or equivalent valid) FIBs with the baseline. Across all experiments (except ones with ECMP enabled), REAL produces FIBs identical to the baseline. For ECMP, the baseline itself may produce different FIBs across multiple runs due to the unstable convergent order, and REAL is confirmed to have equivalent valid FIBs with the baseline. We further check the internal states in RIB, *e.g.*, next-hop, AS-path, and communities, and REAL is also consistent with the baseline. Complex policy decisions such as customer route preference are also correctly maintained. This consistency holds across all supported vendors. We also provide an outline for correctness proof in Appendix F.

REAL recovers control-plane state correctly with message replay. For each failure/recovery event, we export the router's RIB (and relevant neighbor/session state, if available) immediately before the router goes offline and immediately after it is recovered via replay, and compare key attributes (e.g., best-path selection and next-hop set). Across different topologies and vendors, we observe identical (or semantically equivalent in case of ECMP) states, confirming that replay restores control-plane state as intended.

5.2 A Newly Found Firmware Bug

We discovered a race condition in FRRouting's BGP implementation that corrupts peer connections. When a BGP session disconnects, the main thread closes the socket and may immediately reuse its file descriptor for a new connection. However, the BGP I/O thread might still hold a reference to this descriptor from processing pending events. If the I/O thread then performs operations on what it believes is the old connection, it actually affects the new connection, leading to data corruption or unexpected session termination.

Note that simulation tools like Batfish cannot find this bug. REAL performs emulation by executing unmodified control-plane implementations, allowing it to observe implementation-level runtime behaviors such as thread races and file-descriptor life cycle issues. This bug arises at this implementation layer. In contrast, simulation-based tools like Batfish reason about intended routing behavior using abstract protocol models, and are limited to detecting configuration- or policy-level semantic errors. As a result, this runtime race condition is fundamentally out of scope for Batfish.

We have reported this bug to FRRouting's repository, and the maintainer has acknowledged this thread-safety issue[1]. Notably, we have temporarily fixed this issue also by leveraging the shim layer in REAL without modifying the source code of FRRouting. See Appendix D for details.

6 Performance Evaluation

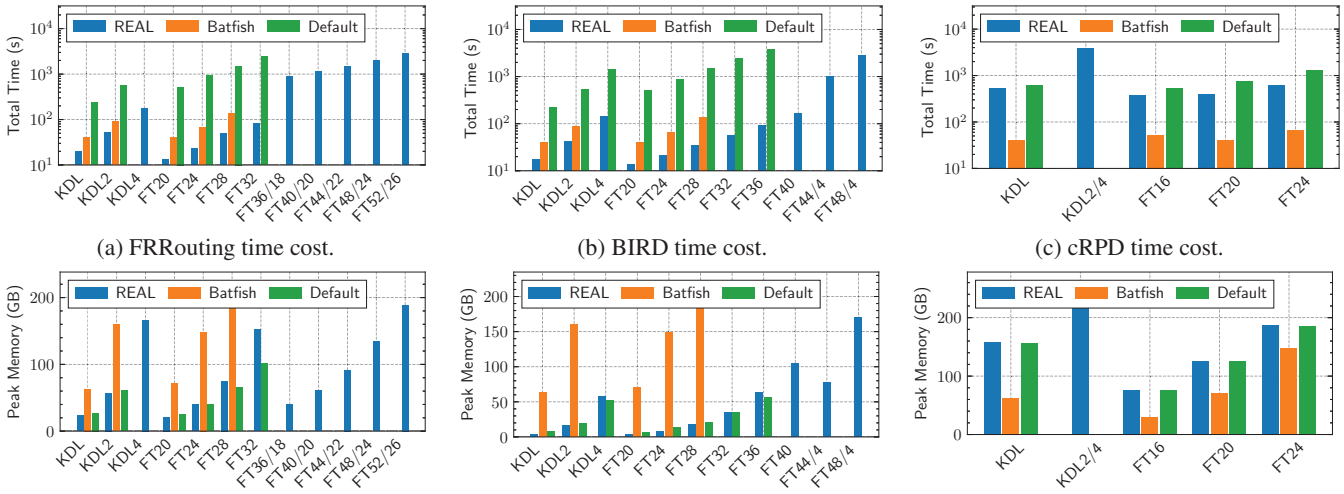
In this section, we evaluate REAL with large networks. In particular, we are interested in the following questions:

1. Can REAL reduce the cost of emulation? With lightweight images, REAL is even faster than Batfish by 3×, and can emulate up to 3,380 routers with a single server.
2. How do optimizations in REAL contribute to its cost reduction? SimDP almost eliminates the bootup cost, accelerating the emulation by up to 41×; the two-phase scheduling reduces the LLC miss rate by 20%; and with iterative convergence, REAL can shave up to 8.3× memory for an ultra-large topology with 4,500 nodes.

6.1 Experimental Setup

Testbed. All experiments are conducted on a cloud server running Linux kernel version 6.8.0, equipped with 64-core CPU and 256GB of memory. Docker version 28.4.0 is used for container management in default runtime. All 64 CPU cores are always enabled for all experiments.

[1] https://github.com/FRRouting/frr/issues/17507



(a) FRRouting time cost. (b) BIRD time cost. (c) cRPD time cost.

(d) FRRouting memory cost. (e) BIRD memory cost. (f) cRPD memory cost.

Figure 9: Time and memory cost for emulating different topologies. Absent bar means failure due to out of memory.

Topologies and configurations. We involve two types of topologies: Fattree with different k, and KDL (754 nodes) from Topology Zoo [1]. For larger WAN topologies, we connect multiple KDL topologies with extra links. We use FT$n/p$ to represent a Fattree $k = n$ topology partitioned into p parts, and KDLm/p to represent a KDL topology extended for m times and partitioned into p parts.

For Fattree topologies, each edge node is configured with $k/2$ prefixes, while aggregation and core nodes do not advertise any prefixes. Aggregation nodes within the same pod share the same AS number, which differs across pods, while all core nodes share a single AS number [32]. ECMP is enabled on all nodes, allowing up to 64 equal-cost paths. For WAN topology, each node has 10 prefixes to advertise, and each node is assigned a unique AS number. We run BGP in all topologies, with simple all-pair reachability configurations.

Approaches for comparison. We involve Linux default runtime and latest Batfish as a comparison to REAL. For Linux runtime, containers are connected with `veth-pair`. To prevent ARP broadcast storms, each node's ARP table is statically configured with the IP addresses of its peers. For Batfish, 1G of stack and 256G of memory are allocated. The total time taken by Batfish is measured from its initialization to the completion of the data plane calculation, excluding any further dataflow analysis and verification.

6.2 End-to-End Performance

We evaluate REAL across three container images (FRRouting, BIRD, and cRPD) on various topologies. REAL enables SimDP and two-phase scheduling by default, with iterative convergence activated for partitioned topologies (FT$n/p$ and KDLm/p). Batfish does not directly support BIRD configurations, but the simulation cost for all-pair reachability across different vendors would be quite similar, so we use FRRouting's results as a reference. Figure 9 presents the time and memory costs of all approaches.

Default runtime suffers from systemic inefficiencies. The Linux default runtime exhibits prohibitive costs across all experiments, requiring at least an order of magnitude more time than both REAL and Batfish. For FRRouting on FT32, it takes 2,466s versus REAL's 83s, with the majority spent on `veth-pair` creation (see Figure 10). While consuming similar memory to REAL, it fails entirely for topologies beyond FT32 due to its always-residency strategy.

REAL outperforms simulation with lightweight containers. For FRRouting and BIRD, REAL consistently achieves the fastest emulation across all tested topologies. For FRRouting with FT28, REAL completes in 49s compared to Batfish's 138s (3× faster). Surprisingly, Batfish's memory consumption also exceeds that of lightweight containers, exceeding 256GB for simulating FT32, while REAL uses only 151GB. This counterintuitive result stems from Batfish's Java-based implementation, which maintains redundant data structures (*e.g.*, huge number of references to routes). We also blame Java's garbage collection for the high memory cost [43].

Heavy vendor images remain tractable. Commercial router images like cRPD impose significantly higher resource demands than open-source alternatives due to heavier inherent workload, *e.g.*, logging system and licensing system. On FT24, cRPD consumes 187GB memory, and exceeds memory of our testbed for FT28. Notably, Batfish is much faster than REAL with cRPD (9× on average). Despite cRPD's inherent overhead, REAL reduces the end-to-end emulation time by 2× compared to the default runtime, from 1,308s to 625s, demonstrating that boot-up acceleration dominates even for resource-intensive containers. We believe that vendors are motivated to further lightweight their products so that customers can conduct control plane emulation at lower cost.

Iterative convergence enables large-scale emulation. When

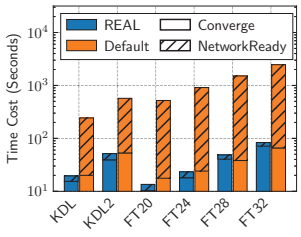


Figure 10: REAL almost eliminates cost of network build-up.

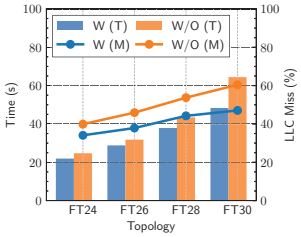


Figure 11: Time cost (T) and LLC miss rate (M) with and w/o two-phase scheduling.

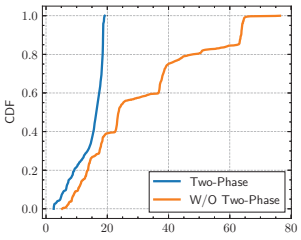


Figure 12: Peer connection buildup time is reduced with two-phase scheduling.

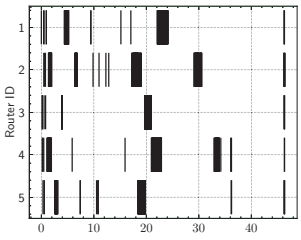


Figure 13: Context switches are reduced by run-to-Idle compared to Figure 2.

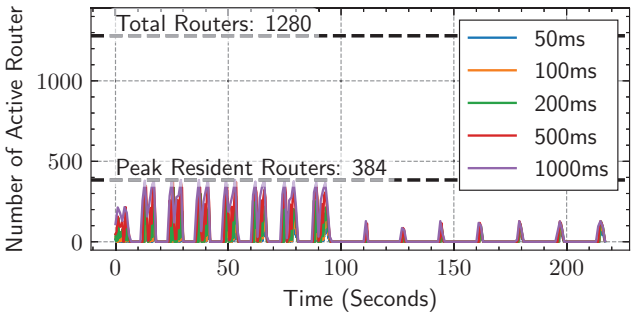


Figure 14: Partitioned reconvergence largely reduced the number of active routers compared to Figure 3.

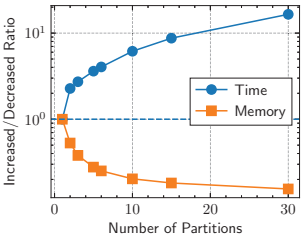


Figure 15: Time-memory trade-off with #partitions.

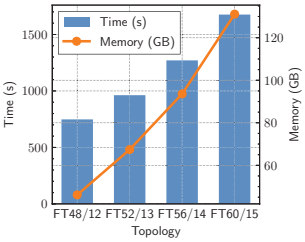


Figure 16: Iterative convergence on ultra-large topo.

memory becomes the bottleneck, REAL with iterative convergence successfully completes emulations that would otherwise fail. On FT52 with 3,380 nodes, REAL completes in 2,835s while iteratively using 189GB memory. As a comparison, FT52 without iterative convergence would require 1,716GB of memory based on our estimation (bridge memory + partition memory $\times$ #partitions). This 9$\times$ memory reduction enables emulation at scales previously impossible on commodity hardware.

6.3 Microbenchmarks

We evaluate the proposed mechanism of REAL. We only involve FRRouting in these experiments.

Simulated data plane. We compare default runtime with REAL to show the reduction of boot-up time. In this comparison, REAL disables two-phase scheduling and iterative convergence. Figure 10 shows that REAL has almost completely removed the overhead of Network-Ready, resulting in a 41$\times$ speedup compared to the default runtime.

Scheduled execution. Figure 11 shows that the two-phase scheduling decreases LLC miss rate from 60% to 47%, and results in a 20% convergence speedup compared to the default runtime. Note that the performance advantage of two-phase scheduling would enlarge as the topology scales, since routers would preempt CPUs more frequently in such large topology.

Figure 12 shows the CDF for establishing connections when emulating FT32. REAL buffers the route updates at first, such that all 16K connections can be established within 20s. On the contrary, the default fair scheduling took 80s in this phase, because it builds up connections and processes route updates in an interleaved way.

Figure 13 shows event diagrams of five sampled nodes when emulating FT32. Compared to the baseline (Figure 2), the two-phase plus run-to-`Idle` scheduling almost eliminates wasteful context switches, and greatly reduces the unnecessary process awakenings.

Iterative Convergence. REAL exploits temporary idleness (§2.3) to reduce peak memory usage with iterative convergence. Figure 14 shows the number of active routers when emulating FT32 partitioned into 8 parts. Iterative convergence guarantees that no more than 384 routers may be online at the same time, as confirmed in Figure 14. This number is largely reduced for each time window shown, compared to Figure 3.

Iterative convergence trades off time for memory. To evaluate such trade-off, we divide pods in FT30 with different number of partitions, and measure their convergence time and memory usage. Figure 15 shows that the time increases from 2.28$\times$ $\rightarrow$ 16.5$\times$ as the number of partitions grows from 2 to 30, while the memory usage decreases by at most 6.4$\times$.

Table 1 details the iterative convergence performance. Our heuristic finds vertex cuts in under 1.5s even for large topologies. All Fattrees converge in 3 iterations, instead of the ideal 2 rounds, because we need another iteration to ensure the whole network converges, which is fast because no router will propagate any route. We also report that WAN topologies require 12-13 iterations due to their less structured nature.

Message-based snapshots dramatically reduce overhead: instead of saving full container memory (hundreds of GB), REAL records only ~8GB of routing messages in memory

Table 1: Microbenchmarks for iterative convergence.

Topo	Partition Time	Iteration Count	Converge Time	Snapshot Size	Restore Time
FT36/2	1.4s	3	268.8s	8.8GB	87.2s
FT36/3	1.4s	3	318.4s	8.3GB	105.4s
FT36/6	1.4s	3	427.5s	8.3GB	138.7s
KDL/4	0.05s	12	791.0s	1.8GB	375.6s
KDL2/4	0.15s	13	1181.0s	6.5GB	632.9s

for FT36 (1620 nodes). Restoring via message replay takes 87-633s in total for multiple iterations. Compared to barely usable disk-based checkpoints, this enables practical iterative convergence at scale.

Handling ultra-large topology with multi-host emulation. We tap the potential of REAL on ultra-large topologies. In these experiments, we leverage the multi-host scalability of REAL with four identical cloud servers, which amounts to 256 cores and 1TB memory. We divide the Fattree topologies into $k/4$ partitions. Figure 16 indicates that REAL can emulate FT60 (a 4,500-node topology) in approximately 1,675s with 494GB memory by dividing it into 15 partitions. We estimate that the memory required by FT60 in default runtime would be ∼4TB. Each host uses three TCP connections to carry 20,736-40,500 inter-host connections with other hosts, whose total bandwidth ranges from 91Mbps to 175Mbps, and does not constitute a performance bottleneck.

7 Related Work and Discussion

We summarize prior works that relate to REAL.

Control plane emulation. Network emulation replicates production environments to validate complex network behaviors, often by running unmodified router and switch images. Early systems like GNS3 [11] and EVE-NG [3] used device virtualization with commercial firmware, ensuring protocol accuracy. Lightweight container-based systems such as Docker-Topo [2], Mininet [31], and Containerlab [8] enabled rapid prototyping. Prior systems such as CrystalNet [33] and Crescent [27] demonstrate that emulation with real router software is effective for production usage. However, emulating medium- to large-scale networks incurs significant resource overheads, with increasing CPU and memory costs [27, 33, 35, 48]. Studies on CloudLab [24] show nonlinear latency growth as the scale exceeds thousands of nodes, even when emulating only part of the network.

Control Plane Simulation. Simulation-based control plane verifiers (CPVs) simulate routing protocols to compute route tables and generate forwarding rules [20, 25, 40, 50]. These tools are useful for control plane management tasks like verification, and can efficiently handle large networks with commodity hardware [20]. However, as mentioned in §1, they risk fidelity issues, as simulated logic may differ from real routers, potentially missing VSBs or firmware bugs [27, 48].

There are also analysis-based CPVs that verify control plane invariants without deriving forwarding rules, relying on customized models such as graphs [18, 28] or SMT constraints [19]. These tools are efficient but disallow operators to manage and observe the real FIBs.

Discussions. We discuss the future directions of REAL.

(1) High-fidelity data plane. SimDP omits packet forwarding behavior to avoid network buildup overhead while preserving control-plane I/O semantics. As a system designed for control-plane emulation, REAL is not intended for data-plane debugging or L2/L3 forwarding validation (*e.g.*, ping/traceroute), which is unnecessary for control plane emulation (§2.3). Extending SimDP with higher-fidelity data-plane functionality for such usage is an important direction for future work.

(2) Supporting VM-based router images. The host machine cannot intercept the syscalls inside the VM, and thus fails to simulate the data plane for the router processes inside the VM. One possible solution is to log into the VM and load shim layer inside it, and connect with other routers through shared memory. But this requires pre-knowledge on the routers' VM, which is not always available. Today, most vendors provide their router images as a container, so we believe that it will not harm the practicality of REAL.

(3) Broader lessons beyond routing. REAL illustrates a general principle: when system correctness is determined primarily by control logic and message-level semantics rather than full data-plane fidelity, substantial performance gains are possible by simulating the runtime while executing unmodified application logic. Beyond routing, similar control-heavy dynamics appear in distributed consensus protocols [30, 37] and cloud orchestration control loops [4, 41]. Exploring the design of REAL in these systems is left as future work.

8 Conclusion

We presented REAL, a novel runtime that emulates control planes with the fidelity of unmodified router images but at the efficiency of simulation. REAL introduces a lightweight data plane, a semantics-aware scheduler, and iterative convergence, enabling scalable and resource-efficient emulation. By reconciling high accuracy with low cost, REAL provides a practical foundation for validating large and complex networks.

This work does not raise any ethical issues.

Acknowledgements. We would like to thank the anonymous NSDI reviewers and our shepherd Ryan Beckett for their valuable feedback. Hao Li is the corresponding author. This work is partially supported by the National Natural Science Foundation of China (No. 62572381, No. 62272382 and No. 62372363).

References

[1] The Internet Topology Zoo. https://topology-zoo.org/, 2013.

[2] Docker-topo. https://github.com/networkop/docker-topo, 2018.

[3] EVE-NG. https://www.eve-ng.net/, 2024.

[4] Kubernetes documentation: Controllers. https://kubernetes.io/docs/concepts/architecture/controller/, 2024.

[5] Checkpoint/Restore In Userspace. https://criu.org/, 2025.

[6] Cisco IOS XRd vRouter. https://www.cisco.com/site/us/en/products/networking/routers/ios-xrd/index.html, 2025.

[7] Containerized routing protocol daemon | juniper. https://www.juniper.net/us/en/products/routers/containerized-routing-protocol-daemon-crpd.html, 2025.

[8] Containerlab. https://containerlab.dev/, 2025.

[9] F-Stack. https://www.f-stack.org/, 2025.

[10] FRRouting. https://frrouting.org/, 2025.

[11] GNS3. https://www.gns3.com/, 2025.

[12] GoBGP. https://github.com/osrg/gobgp, 2025.

[13] Kmesh. https://kmesh.net/, 2025.

[14] Nokia Service Router Linux (SR Linux). https://github.com/nokia/srlinux-container-image, 2025.

[15] The BIRD Internet Routing Daemon. https://bird.network.cz/, 2025.

[16] VRnetlab. https://github.com/vrnetlab/vrnetlab, 2025.

[17] REAL NSDI'26 artifact evaluation repository. https://github.com/ants-xjtu/REAL-artifact-evaluation/tree/nsdi26-ae, 2026.

[18] Anubhavnidhi Abhashkumar, Aaron Gember-Jacobson, and Aditya Akella. Tiramisu: Fast and general network verification. In *USENIX NSDI*, 2020.

[19] Ryan Beckett, Aarti Gupta, Ratul Mahajan, and David Walker. A general approach to network configuration verification. In *ACM SIGCOMM*, 2017.

[20] Matt Brown, Ari Fogel, Daniel Halperin, Victor Heorhiadi, Ratul Mahajan, and Todd Millstein. Lessons from the evolution of the Batfish configuration analysis tool. In *ACM SIGCOMM*, 2023.

[21] Julia Chuzhoy, Yu Gao, Jason Li, Danupon Nanongkai, Richard Peng, and Thatchaphol Saranurak. A deterministic algorithm for balanced cut with applications to dynamic connectivity, flows, and beyond. In *IEEE FOCS*, 2020.

[22] Matthew L. Daggitt, Alexander J. T. Gurney, and Timothy G. Griffin. Asynchronous convergence of policy-rich distributed bellman-ford routing protocols. In *ACM SIGCOMM*, 2018.

[23] Wenliang Du, Honghao Zeng, and Kyungrok Won. Seed emulator: an internet emulator for research and education. In *ACM HotNets*, 2022.

[24] Dmitry Duplyakin, Robert Ricci, Aleksander Maricq, Gary Wong, Jonathon Duerig, Eric Eide, Leigh Stoller, Mike Hibler, David Johnson, Kirk Webb, Aditya Akella, Kuangching Wang, Glenn Ricart, Larry Landweber, Chip Elliott, Michael Zink, Emmanuel Cecchet, Snigdhaswin Kar, and Prabodh Mishra. The design and operation of CloudLab. In *USENIX ATC*, 2019.

[25] Ari Fogel, Stanley Fung, Luis Pedrosa, Meg Walraed-Sullivan, Ramesh Govindan, Ratul Mahajan, and Todd Millstein. A general approach to network configuration analysis. In *USENIX NSDI*, 2015.

[26] Free Software Foundation. ld(1) - linux man page. https://linux.die.net/man/1/ld, 2026. Accessed: 2026-03-14.

[27] Zhaoyu Gao, Anubhavnidhi Abhashkumar, Zhen Sun, Weirong Jiang, and Yi Wang. Crescent: Emulating heterogeneous production network at scale. In *USENIX NSDI*, 2024.

[28] Aaron Gember-Jacobson, Raajay Viswanathan, Aditya Akella, and Ratul Mahajan. Fast control plane analysis using an abstract representation. In *ACM SIGCOMM*, 2016.

[29] EunYoung Jeong, Shinae Wood, Muhammad Jamshed, Haewon Jeong, Sunghwan Ihm, Dongsu Han, and KyoungSoo Park. mTCP: a highly scalable user-level TCP stack for multicore systems. In *USENIX NSDI*, 2014.

[30] Leslie Lamport. The part-time parliament. *ACM Trans. Comput. Syst.*, 16(2):133–169, May 1998.

[31] Bob Lantz, Brandon Heller, and Nick McKeown. A network in a laptop: rapid prototyping for software-defined networks. In *ACM HotNets*, 2010.

[32] P Lapukhov and A Premji. RFC 7938: Use of BGP for routing in large-scale data centers, 2016.

[33] Hongqiang Liu, Yibo Zhu, Jitu Padhye, Jiaxin Cao, Sri Tallapragada, Nuno Lopes, Andrey Rybalchenko, Guohan Lu, and Lihua Yuan. Crystalnet: Faithfully emulating large production networks. In *ACM SOSP*, 2017.

[34] Yadong Liu, Yunming Xiao, Xuan Zhang, Weizhen Dang, Huihui Liu, Xiang Li, Zekun He, Jilong Wang, Aleksandar Kuzmanovic, Ang Chen, and Congcong Miao. Unlocking ECMP programmability for precise traffic control. In *USENIX NSDI*, 2025.

[35] Yunzhuo Liu, Junchen Guo, Bo Jiang, Pengyu Zhang, Xiaoqing Sun, Yang Song, Wei Ren, Zhiyuan Hou, Biao Lyu, Rong Wen, Shunmin Zhu, and Xinbing Wang. Understanding network startup for secure containers in multi-tenant clouds: Performance, bottleneck and optimization. In *Proceedings of the 2024 ACM on Internet Measurement Conference*, IMC '24, page 635–650, New York, NY, USA, 2024. Association for Computing Machinery.

[36] Tie Ma, Long Luo, Hongfang Yu, Xi Chen, Jingzhao Xie, Chongxi Ma, Yunhan Xie, Gang Sun, Tianxi Wei, Li Chen, Yanwei Xu, and Nicholas Zhang. Klonet: an Easy-to-Use and scalable platform for computer networks education. In *USENIX NSDI*, 2024.

[37] Diego Ongaro and John Ousterhout. In search of an understandable consensus algorithm. In *USENIX ATC*, 2014.

[38] Kaifei Peng, Yanbiao Li, Wenbin Li, Xian Yu, and Gaogang Xie. SplitNN: Single-machine network emulation at scale with minute-level construction of 10k-node virtual networks. In *ACM APNet*, 2025.

[39] Shahrooz Pouryousef, Lixin Gao, and Arun Venkataramani. Towards logically centralized interdomain routing. In *USENIX NSDI*, 2020.

[40] Santhosh Prabhu, Kuan-Yen Chou, Ali Kheradmand, P Godfrey, and Matthew Caesar. Plankton: Scalable network configuration verification through model checking. In *USENIX NSDI*, 2020.

[41] Malte Schwarzkopf, Andy Konwinski, Michael Abd-El-Malek, and John Wilkes. Omega: flexible, scalable schedulers for large compute clusters. In *ACM EuroSys*, 2013.

[42] Ranjan Sarpangala Venkatesh, Till Smejkal, Dejan S. Milojicic, and Ada Gavrilovska. Fast in-memory CRIU for docker containers. In *ACM MemSys*, 2019.

[43] Dan Wang, Peng Zhang, Wenbing Sun, Wenkai Li, Xing Feng, Hao Li, Jiawei Chen, Weirong Jiang, and Yongping Tang. S2: A distributed configuration verifier for hyper-scale networks. In *ACM SIGCOMM*, 2025.

[44] Florian Westphal. RTNL mutex, the network stack big kernel lock. In *NetDev 2.2 Conference*, 2017.

[45] Wikipedia. Betweenness centrality. https://en.wikipedia.org/wiki/Betweenness_centrality.

[46] Susan Hares Yakov Rekhter and Tony Li. RFC 4271: A border gateway protocol 4 (BGP-4), 2006.

[47] Hongkun Yang and Simon S Lam. Real-time verification of network properties using atomic predicates. In *IEEE ICNP*, 2013.

[48] Fangdan Ye, Da Yu, Ennan Zhai, Hongqiang Harry Liu, Bingchuan Tian, Qiaobo Ye, Chunsheng Wang, Xin Wu, Tianchen Guo, Cheng Jin, Duncheng She, Qing Ma, Biao Cheng, Hui Xu, Ming Zhang, Zhiliang Wang, and Rodrigo Fonseca. Accuracy, scalability, coverage: A practical configuration verifier on a global WAN. In *ACM SIGCOMM*, 2020.

[49] Yifei Yuan, Fangdan Ye, Yifan Li, Jingkai Zhang, Mengqi Liu, Yuyang Sang, Ruizhen Yang, Duncheng She, Zhiqing Ye, Tianchen Guo, Xiaobo Zhu, Xinji Tang, Li Jia, Zhongyu Guan, Lingpeng Su, Ci Wang, Ruiyang Feng, Shuo Wu, Zhonghui Xie, Cheng Jin, Peng Zhang, Qing Ma, Xianlong Zeng, Dennis Cai, and Ennan Zhai. New evolution of Hoyan: Enhancing scalability, usability, and accuracy for Alibaba's global WAN verification. In *ACM SIGCOMM*, 2025.

[50] Peng Zhang, Aaron Gember-Jacobson, Yueshang Zuo, Yuhao Huang, Xu Liu, and Hao Li. Differential network analysis. In *USENIX NSDI*, 2022.

[51] Peng Zhang, Xu Liu, Hongkun Yang, Ning Kang, Zhengchang Gu, and Hao Li. APKeep: Realtime verification for real networks. In *USENIX NSDI*, 2020.

[52] Lingjun Zhu, Yifan Shen, Erci Xu, Bo Shi, Ting Fu, Shu Ma, Shuguang Chen, Zhongyu Wang, Haonan Wu, Xingyu Liao, Zhendan Yang, Zhongqing Chen, Wei Lin, Yijun Hou, Rong Liu, Chao Shi, Jiaji Zhu, and Jiesheng Wu. Deploying user-space TCP at cloud scale with LUNA. In *USENIX ATC*, 2023.

Appendix A Artifact Appendix

Abstract

This artifact presents the implementation of REAL, a system that supports BGP protocol emulation using FRRouting [10],

BIRD [15], and cRPD [7]. This artifact has been awarded the "Artifacts Available", "Artifacts Functional", and "Artifacts Reproduced" badges via the NSDI 2026 Artifact Evaluation process. We provide comprehensive instructions for environment setup and experiment execution to reproduce the evaluation results discussed in §2.2 and §6.

Scope

This artifact is designed to reproduce all experiments presented in the paper, generating Figures 1–3 and Figures 9–15. Additionally, Figure 16 can be reproduced by manually configuring distributed experiments across multiple machines.

Due to the resource-intensive nature of the full experimental suite (requiring approximately 12 hours), the default launcher is configured to execute a representative subset of experiments that effectively demonstrate key trends. The complete set of experiments requires a machine equipped with at least 256 GB of RAM. Users with limited resources may need to manually reduce the scale of the experiments. While 64 or more CPU cores are recommended for optimal performance, this is not a strict requirement.

REAL is extensible to other BGP implementations, provided they utilize `libc` for TCP/NETLINK socket operations (minor adaptations may be required). However, it does not currently support implementations that bypass `libc` wrappers via direct syscall instructions, such as GoBGP [12].

Note that this artifact is a research prototype and is not intended for production use. It currently supports BGP as the sole control plane protocol (*e.g.*, OSPF is not supported; see Appendix E for extension details) and does not support VM-based vendor images.

Contents

The artifact comprises three main components as listed below.

The Controller. The controller operates as an isolated process outside of any container. It implements the SimDP message relayer (§4.1), the two-phase scheduler (§4.2), and the iterative convergence orchestrator (§4.3).

The Shim. Compiled as a `.so` shared library, the shim is loaded into every containerized process via `LD_PRELOAD`. It handles socket interception and NETLINK message processing for SimDP (§4.1), as well as the idle detection logic for the run-to-idle mechanism (§4.2). Table 2 lists the intercepted `libc` syscall wrappers.

Evaluation Scripts. The artifact includes a suite of scripts for generating configuration files, managing container lifecycles, collecting experimental data, and plotting figures.

Hosting

The complete source code, along with full instructions for building and running the main experiments on REAL, is available in the `nsdi26-ae` branch of the following GitHub repository: https://github.com/ants-xjtu/REAL-artifact-evaluation/tree/nsdi26-ae.

Appendix B Intercepted TCP Sockets

We list the syscalls that the shim layer intercepts for SimDP in Table 2.

Appendix C Finding Optimal Bridge Nodes for Fattree

Fattree, the most widely used topology in data centers, provides a solution for bridge nodes selection by utilizing *core nodes as bridge nodes*. In Fattree, communication between pods is only possible through core nodes, ensuring that each pair of pods is *separated* under this partitioning strategy. This limits the maximum number of simultaneously active nodes to $k^2/4+k$, *i.e.*, the core nodes plus one pod. Additionally, each node only needs to be woken up twice, as all core nodes share the same AS number, preventing any valid route from traversing the core layer more than once.

In the following, we prove that this solution for Fattree is optimal to the partition problem.

Theorem 1. *Using core nodes as bridge nodes in the Fattree topology is the optimal solution for both memory and snapshot efficiency.*

Proof. We prove the theorem by analyzing alternative partitioning strategies and showing that none of them outperform the core-node-as-bridge strategy. Consider a core node c is not a bridge node. Then either all of its neighbors are bridge nodes (Case 1), or all its neighbor belongs to the same partition of c (Case 2).

Case 1. If all of c's neighbors are bridge nodes, this forms a valid partitioning scheme. Moreover, this scheme can be extended to include an additional $k-1$ core nodes into the partition. However, in this case, we trade k always-on core-layer nodes for k always-on aggregation-layer nodes. This substitution is suboptimal because aggregation-layer nodes require significantly more memory than core-layer nodes. The reason is that aggregation-layer nodes store routing information for a larger number of routes compared to core-layer nodes. Thus, this scheme increases memory consumption.

Case 2. If all of c's neighbors belong to the same partition, then all the edge-layer nodes are connected to this partition. Again, there are 2 cases: these edge-layer nodes can either all act as bridge nodes, or belong to the same partition as c. In either scenario, at least all edge-layer nodes ($k^2/2$ in total) must belong to the same partition and be activated simultaneously. Compared to the core-node-as-bridge strategy, this alternative results in a doubling of the number of active

Table 2: The (partial) intercepted `libc` syscall wrappers for simulating TCP sockets.

Application call	**Shim implementation** (`foo_libc` means original libc version of `foo`)
`socket()`	For TCP calls, call `socket_libc(AF_UNIX, SOCK_STREAM, 0)`. For NETLINK calls, pass into kernel and record the returned fd as to be specially processed in later libc calls. Otherwise pass into kernel directly, as well as any other calls on those fds.
`bind(fd, ip:port)`	Record the `ip` and `port` passed in, and call the corresponding `bind_libc()` later on `connect()` or `listen()`.
`connect(fd, d_ip:d_port)`	First, bind the socket using `bind_libc(fd, "/uds/s_ip/d_ip")`, where `s_ip` is either the previously `bind()`-ed IP or inferred from `d_ip` based on the topology. Then, call `connect_libc(fd, "/uds/simdp")` to establish the UDS connection to SimDP. SimDP extracts the source and destination from the UDS address, then attempt to connect to the actual destination, and return the corresponding result.
`listen(fd)`	First, bind the socket using `bind_libc(fd, "/uds/self_ip/")`, then call `listen_libc()`.
`accept(fd, peer_ip_p, addrlen)`	First, pass to the kernel with `peer_ip_p` and `addrlen` replaced by null pointer. Then, read the IP and port from the returned fd (sent by SimDP) and populate the struct pointed to by `peer_ip_p`.
`ioctl/fcntl/getsockopt/setsockopt`	If the semantic applies to unix domain socket, *e.g.*, set the socket to non-blocking mode, pass to kernel. Otherwise recorded in the shim and returns fake response made by shim.
`getsockname/getpeername/getifaddr`	Directly returns fake response made by shim rather than passing to kernel.
`fork/pthread_create/clone`	First passed to kernel. Then re-initialize the shim before returning to application.
`write/writev/sendmsg`	For NETLINK file descriptors, understand requests like `GETLINK`, `NEWROUTE` and construct fake responses for later `recv()`. Otherwise directly passed to kernel.
`read/readv/recvmsg`	Return our injected NETLINK message for NETLINK files. Otherwise directly passed to kernel.
`ppoll/epoll/select`	Return our injected NETLINK message event if there's any. Otherwise passed to kernel with zero timeout, detect `Idle`-ness as in §4.2, and block the entire container if the whole container turns `Idle`. Then pass the original call to kernel. Finally filter out unordered `POLLIN` events on SimDP sockets (see §4.3).
`close`	Fixes FRRoutings thread safely issue as claimed in Appendix D. Directly passed to kernel for other images.
`gettimeofday/clock_gettime`	Return the simulated clock time as in §4.1.

routers. Furthermore, the per-node memory consumption increases significantly because edge-layer routers store routing information for more paths than core-layer routers.

Conclusion. In both cases, alternative partitioning strategies either increase the number of active routers or exacerbate memory consumption per router. Therefore, using core nodes as bridge nodes remains the optimal solution. □

Appendix D Fixing FRRouting Bugs with REAL

Bug discovery and diagnosis. REAL proved essential for both discovering and debugging the FRRouting bug. When the bug was first triggered in a Fattree $k = 24$ topology, the shim layer directly captured the fatal error in `socket()`. By tracing more syscalls, we could observe the exact moment when the main thread's `close()` and `socket()` calls created a reused descriptor that the I/O thread subsequently corrupted.

A temporary mitigation without source modifications. While reporting the race condition to FRRouting's maintainers, we developed a temporary fix that demonstrates REAL's capability for transparent bug mitigation without source code modification. The fix leverages REAL's shim layer to prevent file descriptor reuse during the critical window when the I/O thread might still hold stale references.

Our solution intercepts the `close()` system call to implement atomic descriptor replacement. When the main thread attempts to close a socket descriptor, our redirected `close()` function executes the following sequence: first, it duplicates the original descriptor using `dup(old_fd)` to create `new_fd`. Then, it atomically replaces `old_fd` with a placeholder descriptor pointing to `/dev/null` using `dup2(dumb_fd, old_fd)`. Finally, it closes the duplicate descriptor with `close(new_fd)` and returns its result.

This approach exploits the kernel's atomic guarantees for `dup2()`, which simultaneously closes the target descriptor and replaces it with a duplicate of the source. By replacing `old_fd` with a harmless placeholder, we ensure that any subsequent operations by the I/O thread on the stale descriptor become benign no-ops rather than corrupting a newly allocated connection. The placeholder descriptor remains valid but inert, preventing both use-after-free errors and descriptor reuse conflicts. The fix operates entirely at the system call boundary, requiring no understanding of FRRouting's internal architecture or coordination with the vendor.

Appendix E Extending REAL Beyond BGP: Applicability to OSPF and Other Control Plane Protocols

While our prototype focuses on BGP, the core mechanisms of REAL apply to other control-plane protocols such as OSPF, because they share an event-driven structure and are designed to tolerate diverse message interleavings under eventual delivery. Concretely, extending REAL mainly requires adapting the following components:

Semantics-preserving I/O virtualization. REAL replaces I/O communication primitives with a message-level one that preserves the observable socket semantics. For OSPF, this primarily concerns raw IP/multicast exchanges (e.g., Hellos and flooding). This can be virtualized using topology-aware delivery while keeping packet boundaries and socket semantics,

e.g., datagram-oriented unix domain socket.

Partitioned reconvergence and replay. OSPF's area hierarchy provides a natural partition strategy: non-backbone areas can be treated as edge partitions and the backbone (Area 0) along with ABRs as the core. Replay-based recovery should still work by replaying all the protocol packets to the router.

Note that some protocols/features with tight data-plane coupling may require higher-fidelity data-plane support. A systematic evaluation across additional protocols is left as future work.

Appendix F Correctness Discussion: Timing, Scheduling, and Final Routing Solutions

Trace-based view. Fix a concrete REAL run and its execution trace. For each router, the trace specifies (i) the content and time of routing messages it receives, and (ii) when the router is effectively running (in simulated time). We argue that this trace is simply one valid execution that a normal emulation setup could also realize, if allowed to tweak resource provisioning and message delay.

Time simulation with no starvation baseline. SimDP mainly advances simulated clock on timer expiration, and does so slowly on computation. This matches the operational premise that router control planes are provisioned to run promptly and process routing events quickly relative to protocol timers; a baseline emulator can approximate the same behavior by allocating sufficient CPU (and/or priority) so starvation is negligible.

Two-phase scheduling with message delay shaping. These mechanisms preserve message contents and stream semantics; they only change the time of messages become visible at receivers. For a fixed trace, baseline emulation could delay each routing message, so that it is released at the same simulated time as in the REAL trace. This realizes the same delivery schedule without changing protocol semantics.

Partitioned reconvergence with staged cross-partition propagation. Partitioned reconvergence enforces that edge partitions take effect at the core in phases. A baseline system can mimic this by delaying cross-partition communications, so that updates reach core nodes at the same phase boundaries.

Takeaway. For any specific REAL trace, we can conceptually construct a normal emulation execution that matches its timing and delivery schedule via sufficient CPU provisioning and per-message delay shaping. REAL therefore does not introduce new routing semantics, rather it selects one valid execution among many possible ones.

Finally, we explicitly acknowledge that order-dependent tie-breaking can bias which equivalently valid solution is selected. However, it does not invalidate the final routing solution under the configured semantics.